

Internet of Things

Project Documentation

Eco3DPrint

Francesco Panattoni



University of Pisa
Department of Information Engineering
Academic Year 2025/2026

Contents

1	Introduction	2
2	Environment	4
2.1	Simulation	4
2.2	Contiki-NG	4
2.3	Cooja	4
2.4	Tunslip	5
2.5	Nordic Dongle NRF52840	5
3	Architecture	6
3.1	Ecosystem	6
3.2	IoT Network	6
3.2.1	Protocol Stack	6
3.2.2	Nodes	7
3.3	Cloud Application	8
3.3.1	Backend	8
3.3.2	Frontend	8
3.4	Project Alignment	9
3.4.1	Project Objective	9
3.4.2	Application Protocol	9
3.4.3	Edge Intelligence and Autonomous Decision-Making	10
3.4.4	Hardware Interaction	10
3.4.5	Data Encoding	10
4	Implementation	11
A	Appendix	12

Chapter 1

Introduction

The advent of **3D Printing** has revolutionized manufacturing, prototyping, and personal fabrication by allowing digital models to be transformed into physical objects layer by layer. However, the additive **manufacturing process is inherently susceptible to physical anomalies**. A slight deviation in temperature, extrusion rate, or mechanical calibration can lead to catastrophic print failures. When a printer operates unmonitored for extended periods, these undetected failures result in a significant waste of filament materials, excessive energy consumption, and lost time.

To mitigate this inefficiency, we must look toward modern networking paradigms. The **Internet of Things** (IoT) represents the evolution of the network that extends connectivity to physical objects of daily life. By transforming standalone, "dumb" 3D printers into smart, interconnected assets, we can actively monitor their health and intervene dynamically before resources are wasted.

This report presents ***Eco3DPrint***, *an intelligent monitoring application designed to safeguard the 3D printing process*. An IoT system is composed of four functional levels: devices and sensors that collect data, connectivity that allows data to travel, data processing on the cloud, and a user interface. The architecture is inspired to the "Comprehensive Review on Internet of Things Applications in Power Systems" [1].

Our solution fully embodies this architecture. We deploy specialized, network-connected dongles equipped with accelerometers and voltage sensors directly onto the printers. As the machine operates, these sensors continuously monitor the physical dynamics of the extruder and the build plate, alongside the power drawn by the system. The collected data is fed into a localized *Machine Learning (ML) model* trained to distinguish between normal operational vibrations and the erratic, anomalous patterns indicative of a failing print.

Upon detecting an anomaly, the system automatically halts the printing process to prevent further material and energy waste. Users manage the entire ecosystem through a comprehensive software suite (a *Cloud backend* and a *User Application frontend*) which facilitates the direct transfer of STL files to the machines via the network. Furthermore, *the application provides transparent, analytical tracking of daily, weekly, and monthly energy consumption, explicitly differentiating between "well-used" energy (successful prints) and "wasted" energy (failed attempts)*.

From a technical standpoint, deploying **constrained embedded devices** at the edge of the network is a deliberate and crucial architectural choice. While a high-powered, general-purpose computer could theoretically monitor a printer, this application aligns with the principles of the *Industrial Internet of Things (IIoT)*, where efficiency and scalability are paramount.

Using constrained devices enables cost-effective retrofitting, allowing existing 3D printers without built-in monitoring to be integrated into an IoT ecosystem through inexpensive embedded dongles, avoiding costly hardware replacement. It also improves network efficiency by processing high-frequency telemetry locally thereby reducing bandwidth usage while relying on lightweight protocols such as *CoAP* and *MQTT* instead of heavier alternatives like *HTTP*.

Finally, constrained devices support net-positive energy savings. Since they consume far less power than traditional computing systems, they ensure that the energy required for monitoring does not outweigh the savings gained from preventing failed prints, maintaining both economic and environmental viability.

Chapter 2

Environment

2.1 Simulation

This project is developed as part of an academic simulation within the Internet of Things course, and is therefore designed to operate within the controlled environment and assumptions defined by the professors, which are outlined briefly below.

2.2 Contiki-NG

Contiki-NG [2] represents a sophisticated, open-source operating system explicitly architected for resource-constrained embedded devices within the Internet of Things ecosystem.

At the foundation of its architecture lies a highly modular, event-driven kernel that operates utilizing Protothreads. This specialized programming model facilitates memory-efficient cooperative multithreading without incurring the heavy overhead typically associated with per-thread stacks, thereby allowing the entire operating system to function optimally within remarkably tight hardware constraints, often requiring as little as 10 kilobytes of random-access memory and a total code footprint of approximately 100 kilobytes.

2.3 Cooja

Cooja is an advanced, Java-based open-source network simulator inherently integrated within the Contiki and Contiki-NG ecosystems, specifically engineered to emulate the intricate dynamics of IoT and wireless sensor networks. It supports cross-level simulation, allowing analysis from network topology down to operating system behavior and machine code execution. It can emulate hardware using tools like MSPSim, enabling real embedded code to run as it would on devices such as MSP430-based nodes.

Cooja offers different mote types for testing: fast "Cooja motes" for high-level simulation and emulated motes for detailed hardware-level validation, including drivers and power management. With a graphical interface for visualizing networks, timelines, and serial output, it helps developers analyze radio interactions, debug protocols, and optimize systems before deploying on real hardware.

2.4 Tunslip

Tunslip is a utility included in Contiki-NG that establishes a virtual SLIP (Serial Line Internet Protocol) link between a border router and a host computer. It is primarily used to bridge an IoT network with the host's networking stack by creating a tunnel interface.

Through this virtual connection, IPv6 packets generated within a simulated or physical sensor network can be forwarded to and from external networks, including the internet. This makes it possible for constrained IoT nodes to communicate with standard IP-based systems as if they were part of the same routed network, facilitating testing and integration of networked applications.

2.5 Nordic Dongle NRF52840

The **nRF52840 Dongle** is a compact, low-cost USB development platform from *Nordic Semiconductor* built around the nRF52840 SoC. It uses a 32-bit ARM Cortex-M4 processor with a hardware FPU running at 64 MHz and includes 1 MB of Flash and 256 KB of RAM for embedded applications.

It supports multiple 2.4 GHz wireless protocols, including Bluetooth 5 (BLE, long range, 2 Mbps, and mesh), IEEE 802.15.4 for Thread and Zigbee, as well as ANT and proprietary protocols. Security is handled by the integrated CryptoCell-310 hardware accelerator for cryptographic operations.

The board is USB-powered, operates from 1.7 V to 5.5 V, and includes a user button, LEDs, and 15 GPIOs accessible via edge pads. It can be programmed and updated via USB and integrates with the nRF Connect for Desktop ecosystem without requiring an external debugger.

Chapter 3

Architecture

3.1 Ecosystem

The system is composed of an **IoT Network** and a **Cloud Application**.

The *IoT Network* consists of a **Border Router** and a set of **Printers**, each connected to a **Dongle** and a set of **sensors**.

The *Cloud Application* serves as the central orchestration hub of the *Eco3DPrint* ecosystem, coordinating communication between the distributed hardware nodes and the **User App interface**.

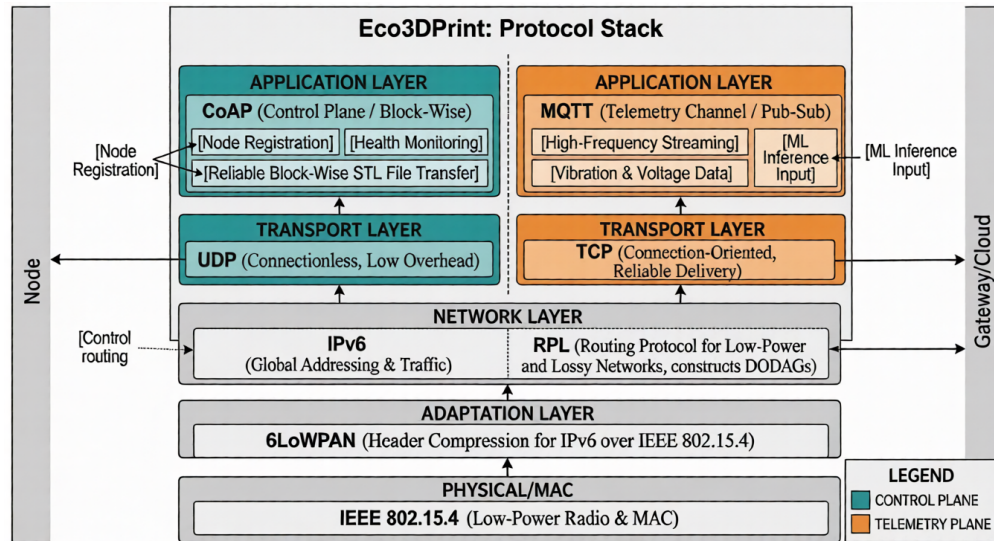
3.2 IoT Network

3.2.1 Protocol Stack

The communication architecture implements a specialized multi-layered stack to bridge physical sensors with the cloud. At the base, **IEEE 802.15.4** provides the low-power radio foundation, while the **6LoWPAN** adaptation layer compresses headers to allow **IPv6** traffic over constrained links. Networking is maintained by **RPL** (Routing Protocol for Low-Power and Lossy Networks), which constructs dynamic routing paths to ensure data reaches the gateway even in lossy wireless environments.

The application layer is split according to functional requirements. **CoAP** (Constrained Application Protocol) is used for control-plane operations, including node registration, health monitoring, and reliable block-wise transfer of **STL files**, following a lightweight request-response model. It runs over **UDP** (User Datagram Protocol), a connectionless transport protocol that minimizes overhead and latency but does not guarantee delivery or ordering. In contrast, **MQTT** (Message Queuing Telemetry Transport) provides a dedicated telemetry channel based on a publish-subscribe architecture to stream high-frequency vibration and voltage data for real-time machine learning inference. It operates over **TCP** (Transmission Control Protocol), a connection-oriented transport protocol that ensures reliable, ordered, and error-checked delivery of data streams.

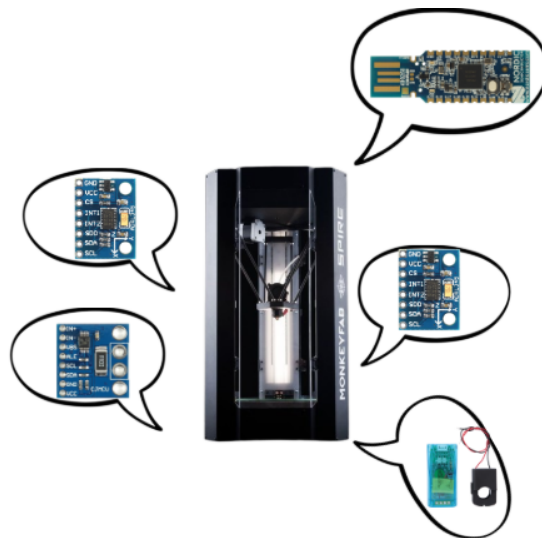
The system utilizes **JSON** (JavaScript Object Notation) as the primary data encoding format for control and telemetry messages. For the transmission of 3D models, the application implements a **CoAP Block-wise transfer mechanism**.



Personalized Protocol Stack of the App

3.2.2 Nodes

The IoT Network consists of a variable number of Printer nodes connected to the Network with a nRF52840 Dongle. These nodes serve as the physical interface of the system, integrating sensors that collect real-time vibration and voltage data, alongside the Printer as an actuator. By running an embedded Machine Learning model, the nodes autonomously stop print jobs when anomalies are detected, improving energy efficiency without external intervention. The border router acts as the critical gateway of the architecture, enabling the translation of traffic between the low-power wireless network and the standard IPv6 infrastructure. This component allows the devices to communicate seamlessly with the Cloud Application.



3D Printer with Dongle and Sensors

3.3 Cloud Application

3.3.1 Backend

The **Cloud Application backend** is built on a multi-threaded Python architecture, and it bridges the resource-constrained IoT network with the graphical User Application (the frontend). The backend is strictly **modular**, separating network communication, business logic, and data persistence into dedicated functional components.

- **Database Access Layer:** Manages persistent data storage in a MySQL database, utilizing Connection Pooling to safely handle concurrent, thread-safe read and write operations;
- **CoAP Server:** Runs over UDP/IPv6 to handle low-frequency control events, including node registration, end-of-print notifications, and shutdown signals. It is implemented using **CoAPthon** [3];
- **MQTT Handler:** Runs as a background subscriber to continuously ingest high-frequency JSON telemetry (vibration and voltage data) during active prints, mapping the data directly to the database. It is implemented using **Paho** [4];
- **Node Monitor:** Maintains real-time fleet health by concurrently pinging active nodes via CoAP /health requests every 60 seconds. It utilizes an Observer pattern to instantly broadcast state changes (*OFFLINE*, *ONLINE*, *PRINTING*) across the system;
- **Print Manager:** Orchestrates the print job queue and handles the reliable transmission of 3D models. It utilizes CoAP Block-wise transfer (Block1) to partition and sequentially send large binary STL files to available nodes;
- **WebSocket Manager:** Provides a full-duplex asynchronous connection to the User App. It pushes live node status updates to the dashboard and processes complex analytical requests to generate daily, weekly, and monthly energy reports.

3.3.2 Frontend

The **Frontend Application (User App)** is a desktop-based graphical interface built using the Python **tkinter framework** [5]. A background daemon thread runs an asyncio event loop to manage a persistent WebSocket connection, enabling continuous data exchange without blocking the interface. Incoming messages are passed via a thread-safe queue to the main GUI thread, which polls, parses, and routes them to the appropriate screen based on their type:

- **3D Printers Dashboard:** Displays a responsive grid of interactive cards for real-time fleet management. It dynamically applies color filters to printer icons to reflect current operational states (*OFFLINE*, *ONLINE*, *PRINTING*) and includes modal dialogs for viewing node details and queueing STL files;
- **Daily Report:** Visualizes the current day's energy metrics, automatically converting raw Joules to kWh. It features high-level global summary cards and scrollable tables (ttk.Treeview) that break down well-used versus wasted energy by individual printer and STL file;
- **Weekly Report:** Aggregates telemetry over the week. It implements a zero-padded data structure to ensure consistent daily rendering and utilizes multiple centered tables to highlight energy consumption trends throughout the week;

- **Monthly Report:** Provides a long-term analytical view with a Year-To-Date (YTD) summary. It displays a month-by-month historical table tracking total print volume, successful energy usage, and energy lost to anomalies.

3.4 Project Alignment

3.4.1 Project Objective

Smart Homes and Buildings: enabling real-time control and energy efficiency in residential or commercial environments.

I choose "*Smart Homes and Buildings*" as a domain for my project. I have enabled a **real-time monitoring application** where I monitor the status of the printers, create a list of STLs to print in real time and monitor the energy consumption of the printers. All of this is **designed to monitor energy efficiency**, supported by a **Machine Learning model** that proactively halts faulty prints before they occur. This system is primarily intended for **commercial environments** where multiple 3D printers operate simultaneously, such as print farms or production labs, where efficiency, reliability, and cost control are critical. However, it is equally applicable in advanced home setups with multiple 3D printers, offering the same benefits of centralized monitoring, energy optimization, and failure prevention on a smaller scale.

3.4.2 Application Protocol

The architecture utilizes **CoAP** as the primary mechanism for the **Eco3DPrint** application. It is used for control-plane operations, including node registration, health monitoring, and file transfers.

CoAP was selected over MQTT for these specific reasons:

- **RESTful Interaction for Registration and Health:** Unlike the publish-subscribe model of MQTT, CoAP is a request-response protocol that operates over UDP. This eliminates the need for the device to maintain a persistent, energy-intensive connection just to signal availability, significantly improving energy efficiency during idle periods;
- **File Transferring:** CoAP is simply better at sending larger objects like files than MQTT;
- **Reduced Protocol Overhead:** Because CoAP does not require the overhead of the TCP three-way handshake for every interaction, it reduces the total number of radio transmissions needed for occasional tasks like registration. Fewer transmissions directly translate to lower power consumption, extending the battery life of the monitoring dongles.

While CoAP is well suited for discrete control tasks, **MQTT** is used exclusively to stream high-frequency sensor data during the active printing phase. Since this phase already represents peak power consumption for the printer, the additional energy overhead of MQTT is relatively negligible. To further optimize resource usage, a 10-second timer is implemented: if no new print job is received, the MQTT connection is automatically closed, minimizing the node's power footprint once peak activity ends.

Furthermore a **Publish–Subscribe model is more efficient for this telemetry plane than a Request–Response approach**. Once connected, it streams large volumes of JSON data with minimal overhead, avoiding per-message acknowledgment costs.

3.4.3 Edge Intelligence and Autonomous Decision-Making

The system runs a **Machine Learning model** directly on constrained nRF52840 nodes, enabling real-time, autonomous responses to local anomalies without relying on the cloud. A C-based implementation performs on-device feature extraction (mean, standard deviation, and peak-to-peak across multiple axes) reducing bandwidth usage and ensuring the printer can be stopped even during temporary connectivity loss.

3.4.4 Hardware Interaction

The project strictly adheres to the mandatory button and LED interaction requirements.

A **physical button** is used for manual overrides, system resets, and confirming print completions, ensuring a human-in-the-loop safety mechanism.

At the same time, **LEDs** provide essential visual feedback through color coding—yellow for initialization, green for online status, and red for failure—allowing an on-site operator to quickly assess device status without a display.

(REWRITE FOR THE DONGLE)

3.4.5 Data Encoding

JSON (JavaScript Object Notation), structured via the **SenML** (Sensor Measurement Lists [6]) data model, is the optimal encoding choice because its minimalist key-value structure drastically reduces packet overhead compared to the verbose tagging required by XML. This standardized approach facilitates a seamless data pipeline between the C-based firmware and the Python backend without the heavy memory or CPU requirements of an XML DOM parser.

For the transmission of STL models, the system implements **CoAP Block-wise transfer** to overcome hardware link limitations by partitioning binary data into sequential 64-byte chunks. This mechanism ensures reliable delivery through application-layer acknowledgments and prevents RAM exhaustion on the constrained nRF52840 nodes by processing only one fragment at a time.

...

Chapter 4

Implementation

...

Appendix A

Appendix

...

Bibliography

- [1] A. Majhi, A. Kumar, and S. Mohanty, “A comprehensive review on internet of things applications in power systems,” *IEEE*, 2024. [Online]. Available: https://www.researchgate.net/publication/383289893_A_Comprehensive_Review_on_Internet_of_Things_Applications_in_Power_Systems
- [2] “Contiki-ng wiki,” 2026. [Online]. Available: <https://github.com/contiki-ng/contiki-ng/wiki>
- [3] “Coapthon.” [Online]. Available: <https://github.com/Tanganelli/CoAPthon>
- [4] “Paho.” [Online]. Available: <https://pypi.org/project/paho-mqtt/>
- [5] “Tkinter.” [Online]. Available: <https://docs.python.org/3/library/tkinter.html>
- [6] “Senml - a lightweight format for representing sensor data.” [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8428.html>